INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	ASPECTOS FORMALES DE ESPECIFICACIÓN Y VERIFICACIÓN				
TÍTULO DE LA PRÁCTICA:	Tipo Map				
NÚMERO DE PRÁCTICA:	06	**AÑO LECTIVO:**	2026B	**NRO. SEMESTRE:**	VIII
FECHA DE PRESENTACIÓN	26/10/26	**HORA DE PRESENTACIÓN**	23:59		
INTEGRANTE (s): Choquehuanca Zapana Hernan Andy				**NOTA:**	
DOCENTE(s): Mg. Roxana Limache Calatayud					

SOLUCIÓN Y RESULTADOS

I. SOLUCIÓN DE EJERCICIOS RESUELTOS

Repositorio con los archivos generados en el laboratorio:

https://github.com/hz07s/AFEV-Lab06.git

[CAPTURA 1 PENDIENTE — repositorio en GitHub (hz07s/AFEV-Lab06). Pegar tras el push.]

Repositorio del laboratorio con las clases TiposMap, Purse, System, PruebaSystem, Diccionario y PruebaDiccionario, y las fuentes del diagrama de clases.

Caso de aplicación: operaciones con el tipo Map

1. Descripción del caso

Calcular de forma manual y validar en el intérprete de VDM++ expresiones con el tipo Map: comprensión de mappings, tipos de dominio y rango, y los operadores dom, rng, munion, ++ y restricción de dominio (<:). La consola no conserva variables entre líneas, por eso cada expresión repite el literal completo del mapping. Para la Parte 2 se creó la clase TiposMap, que declara m1 y m2 con su tipo para que el verificador de tipos los compruebe.

- Parte 1: comprensión de mappings con y sin filtro.
- Parte 2: tipo de m1 (map PAIS to nat) y de m2 (map nat to nat).
- Parte 3: dom, rng, munion, ++ y <: sobre m1 y Europe.
- Parte 4: dom y rng de mappings pequeños, incluido el mapping vacío.
- Casos adicionales: munion con una clave en conflicto, mapping inválido e inverse de un mapping no inyectivo.

2. Comprensión de mappings (Parte 1)

Cálculo manual de (a) { i**2 |-> i/2 | i in set {1,2,4,5} }: se evalúan i**2 e i/2 para cada elemento del conjunto.

i	i**2	i/2	Maplet
1	1	0.5	1 |-> 0.5
2	4	1	4 |-> 1
4	16	2	16 |-> 2
5	25	2.5	25 |-> 2.5

Resultado de (a): {1 |-> 0.5, 4 |-> 1, 16 |-> 2, 25 |-> 2.5}. Las claves no se repiten porque i**2 es distinto para cada i, así que el mapping es válido. En (b) ningún elemento de {1, 2, 4, 5} cumple i < 1, por lo que el resultado es el mapping vacío {|->}.



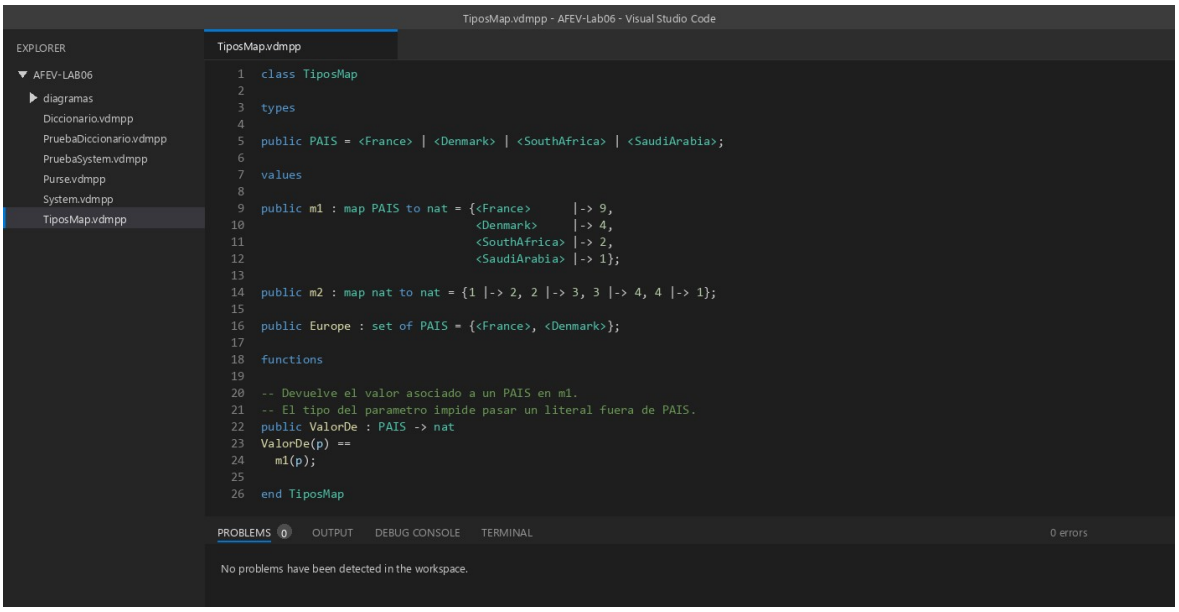
La consola devuelve {16 |-> 2, 1 |-> 0.5, 4 |-> 1, 25 |-> 2.5} para (a) y {|->} para (b): los mismos maplets del cálculo manual, listados en otro orden porque un mapping no tiene orden. El rango contiene reales porque i/2 es una división real.

3. Tipos de m1 y m2 (Parte 2)

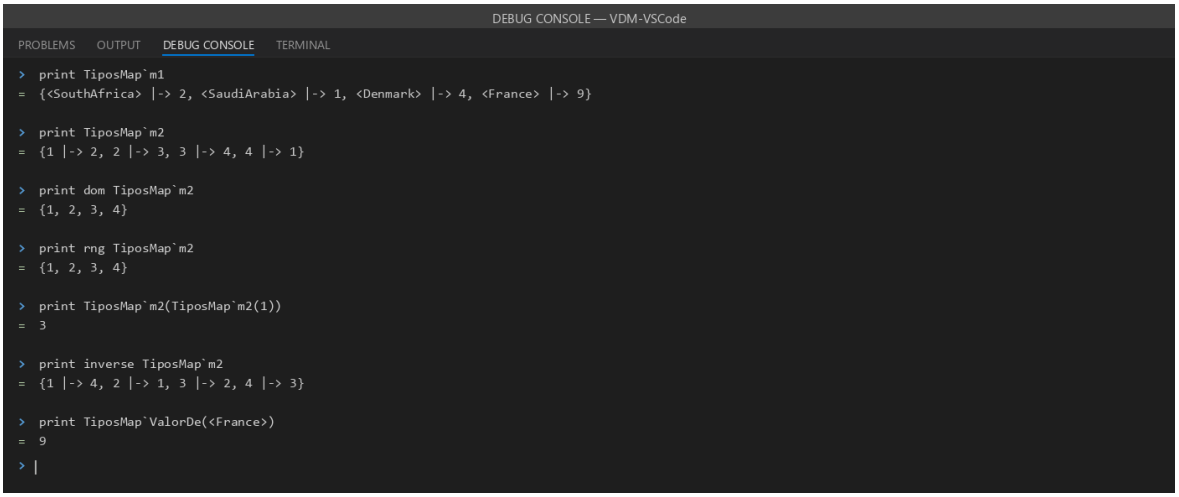
m1 : map PAIS to nat tiene como dominio el tipo enumerado PAIS, que solo admite <France>, <Denmark>, <SouthAfrica> y <SaudiArabia>, y como rango cualquier natural. m2 : map nat to nat admite cualquier natural como clave y como valor. m2 = {1 |-> 2, 2 |-> 3, 3 |-> 4, 4 |-> 1} es además inyectivo, porque ningún valor del rango se repite, y por eso admite inverse.

TiposMap.vdmpp declara PAIS, los valores m1 y m2 con su tipo y la función ValorDe, que recibe

un PAIS y devuelve m1(p). Así el verificador de tipos rechaza cualquier valor fuera de PAIS.

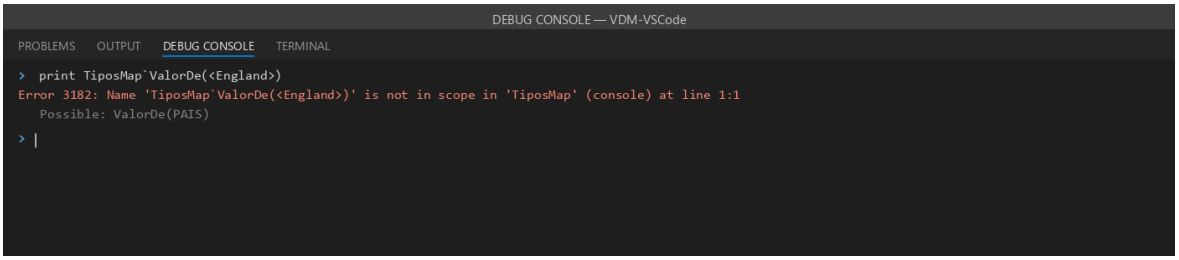


Se aprecia TiposMap.vdmpp con el tipo enumerado PAIS, los valores m1 (map PAIS to nat) y m2 (map nat to nat) y la función ValorDe. La extensión VDM-VSCode no reporta errores de sintaxis ni de tipos.



dom y rng de m2 son {1, 2, 3, 4}. m2(m2(1)) aplica dos veces el map: m2(1) = 2 y m2(2) = 3. inverse devuelve {1 |-> 4, 2 |-> 1, 3 |-> 2, 4 |-> 3} porque m2 es inyectivo, y ValorDe(<France>) devuelve 9.

Caso límite: valor fuera del dominio PAIS



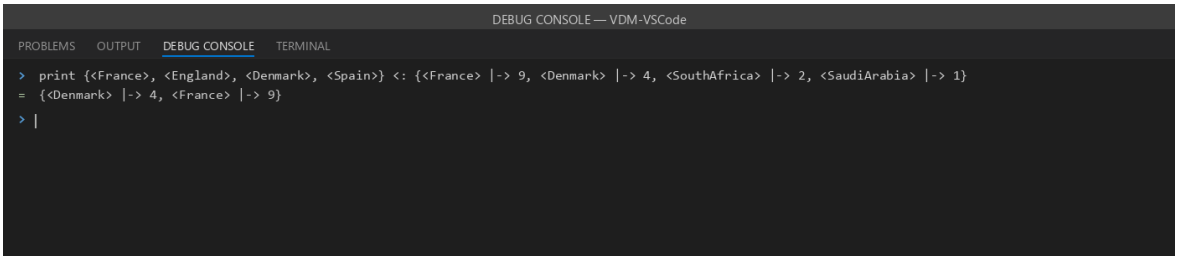
<England> no pertenece al tipo PAIS, por lo que el verificador rechaza la expresión antes de evaluarla. VDMJ no lo reporta como un error de tipos corriente, sino como Error 3182: Name 'TiposMap`ValorDe(<England>)' is not in scope, y sugiere la única signatura existente, ValorDe(PAIS). Esto confirma que el dominio de m1 está limitado a los cuatro literales de PAIS.

4. Operaciones dom, rng, munion, ++ y <: (Parte 3)

Expresión	Resultado esperado	Motivo
dom m1	{<France>, <Denmark>, <SouthAfrica>, <SaudiArabia>}	Claves de m1.
rng m1	{1, 2, 4, 9}	Valores asociados, como conjunto.
m1 munion {<England> |-> 3}	m1 más el maplet <England> |-> 3	No hay claves en común, la unión está definida.
m1 ++ {<France> |-> 8, <England> |-> 4}	<France> pasa a 8 y se agrega <England> |-> 4	El mapa de la derecha sobrescribe.
Europe <: m1	{<France> |-> 9, <Denmark> |-> 4}	Solo quedan las claves de m1 que están en Europe.

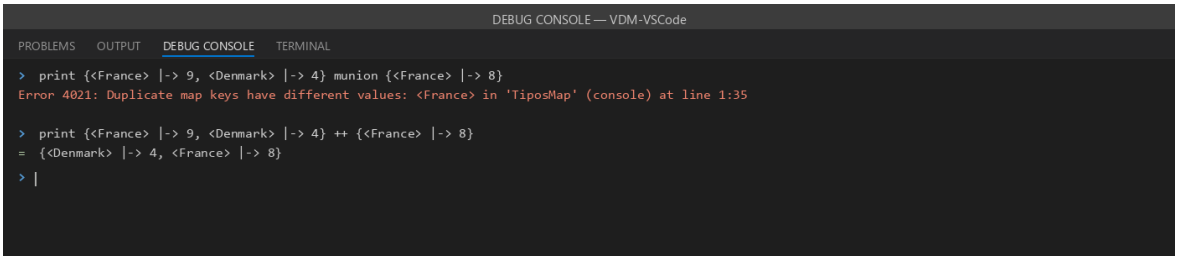


Los resultados coinciden con la tabla. En ++ el valor de <France> cambia de 9 a 8 sin error. El orden en que la consola lista los maplets puede variar, porque un mapping no tiene orden.



La restricción conserva solo <France> y <Denmark>. <England> y <Spain> están en Europe pero no en dom m1, por eso no aparecen.

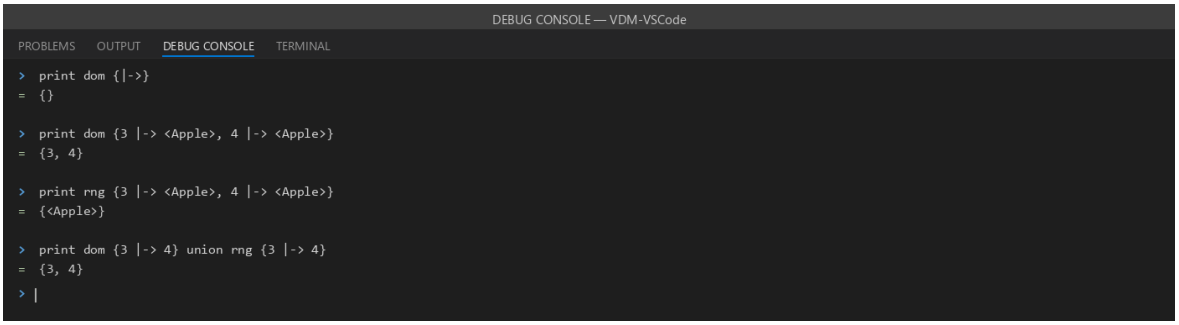
Caso límite: munion con una clave en conflicto



munion falla porque <France> está en ambos mapas con valores distintos (9 y 8): es una operación parcial. Con ++ la misma expresión devuelve {<France> |-> 8, <Denmark> |-> 4}, porque prevalece el mapa de la derecha.

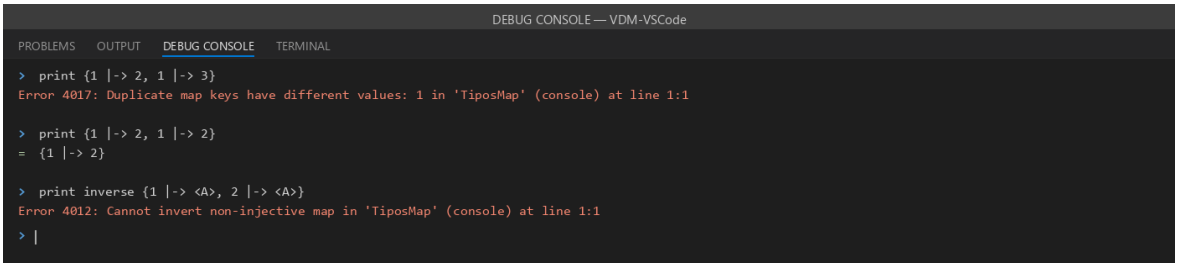
5. Resultado de operaciones (Parte 4)

Expresión	Resultado manual	Justificación
dom {|->}	{}	El mapping vacío no tiene claves.
dom {3 |-> <Apple>, 4 |-> <Apple>}	{3, 4}	Dos claves distintas.
rng {3 |-> <Apple>, 4 |-> <Apple>}	{<Apple>}	rng es un conjunto: el valor repetido cuenta una vez.
dom {3 |-> 4} union rng {3 |-> 4}	{3, 4}	{3} union {4}.



Los cuatro resultados coinciden con el cálculo manual. El tercero confirma que un mapping no inyectivo es válido y que rng elimina la repetición de <Apple>.

Mappings inválidos y no inyectivos (Figura 1 de la guía)



{1 |-> 2, 1 |-> 3} produce un error porque la clave 1 apunta a dos valores. {1 |-> 2, 1 |-> 2} se acepta y queda {1 |-> 2}, ya que el maplet repetido es idéntico. inverse falla porque el mapping no es inyectivo.

II. SOLUCIÓN DE EJERCICIOS PROPUESTOS

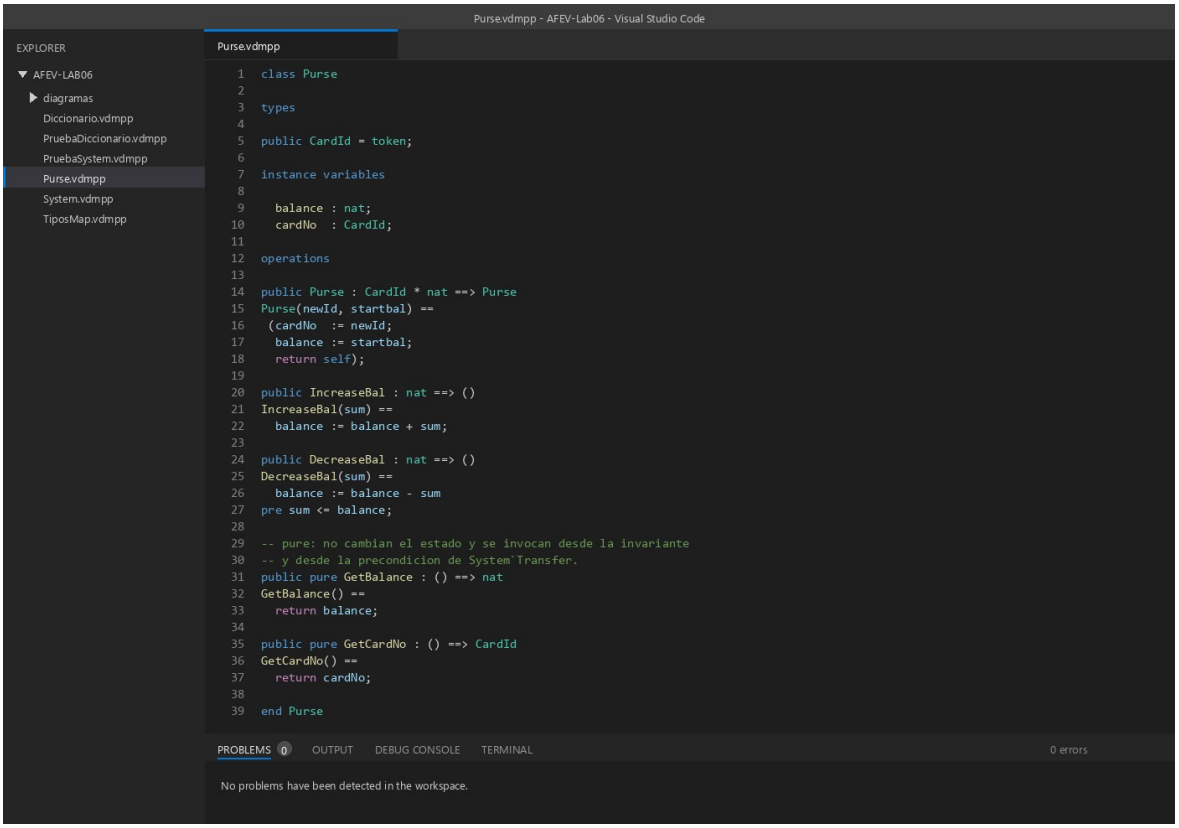
Ejercicio 1: Sistema de Billeteras. Caso de aplicación: clases Purse y System

1. Descripción del caso

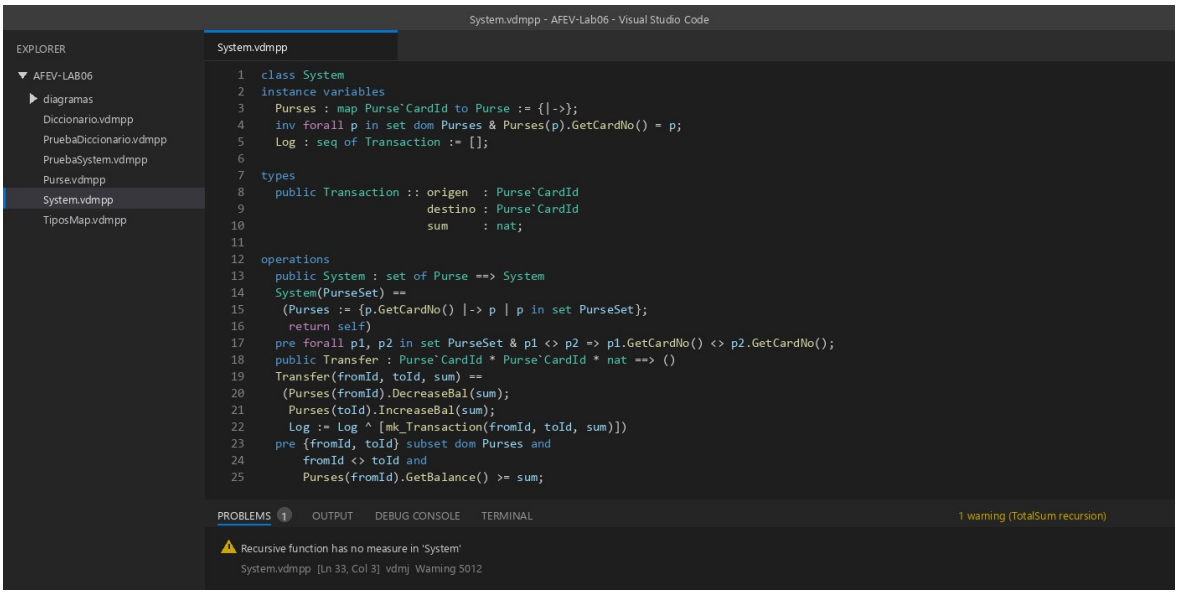
Un sistema administra billeteras (Purse), cada una con un saldo y un número de tarjeta de tipo token. System guarda las billeteras en el map Purses, indexado por número de tarjeta, y registra cada transferencia en la secuencia Log. Se usa el código de la guía; solo se marcaron GetBalance y GetCardNo como pure, porque se llaman desde la invariante y desde la precondición de Transfer.

- Invariante de System: cada clave de Purses coincide con el número de tarjeta de su billetera.
- Precondiciones: tarjetas distintas en el constructor; en Transfer, ambas tarjetas registradas, distintas y con saldo suficiente.
- Incisos: a) operaciones del tipo Map usadas, b) invariante en lenguaje natural, c) diagrama de clases.
- Se añaden la clase PruebaSystem y la operación RegistrarSinValidar, solo para pruebas.

2. Código completo - Purse.vdmpp y System.vdmpp

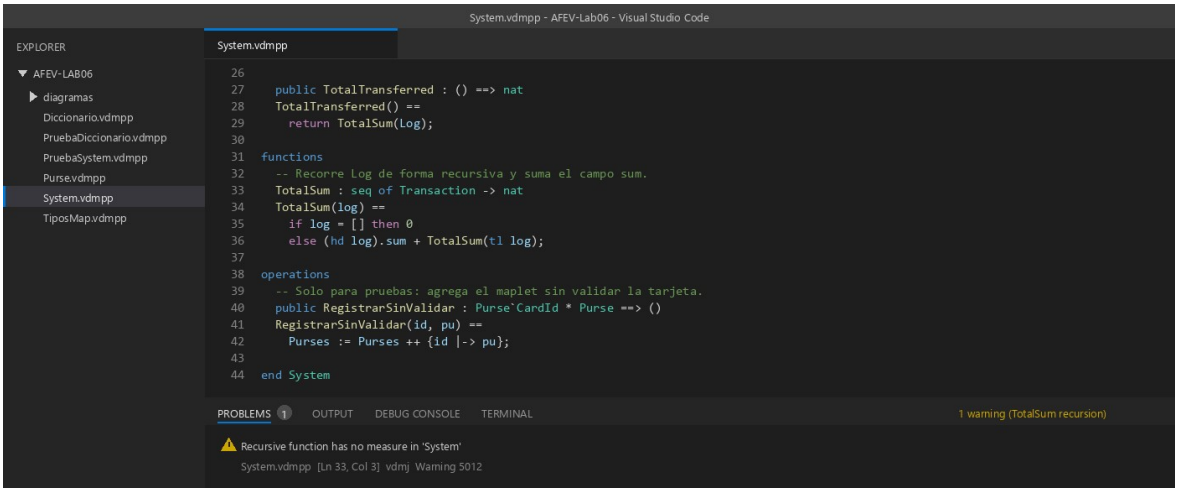


Se aprecia Purse.vdmpp con el tipo CardId (token), las variables balance y cardNo, el constructor y las operaciones IncreaseBal, DecreaseBal, GetBalance y GetCardNo. La extensión no reporta errores.

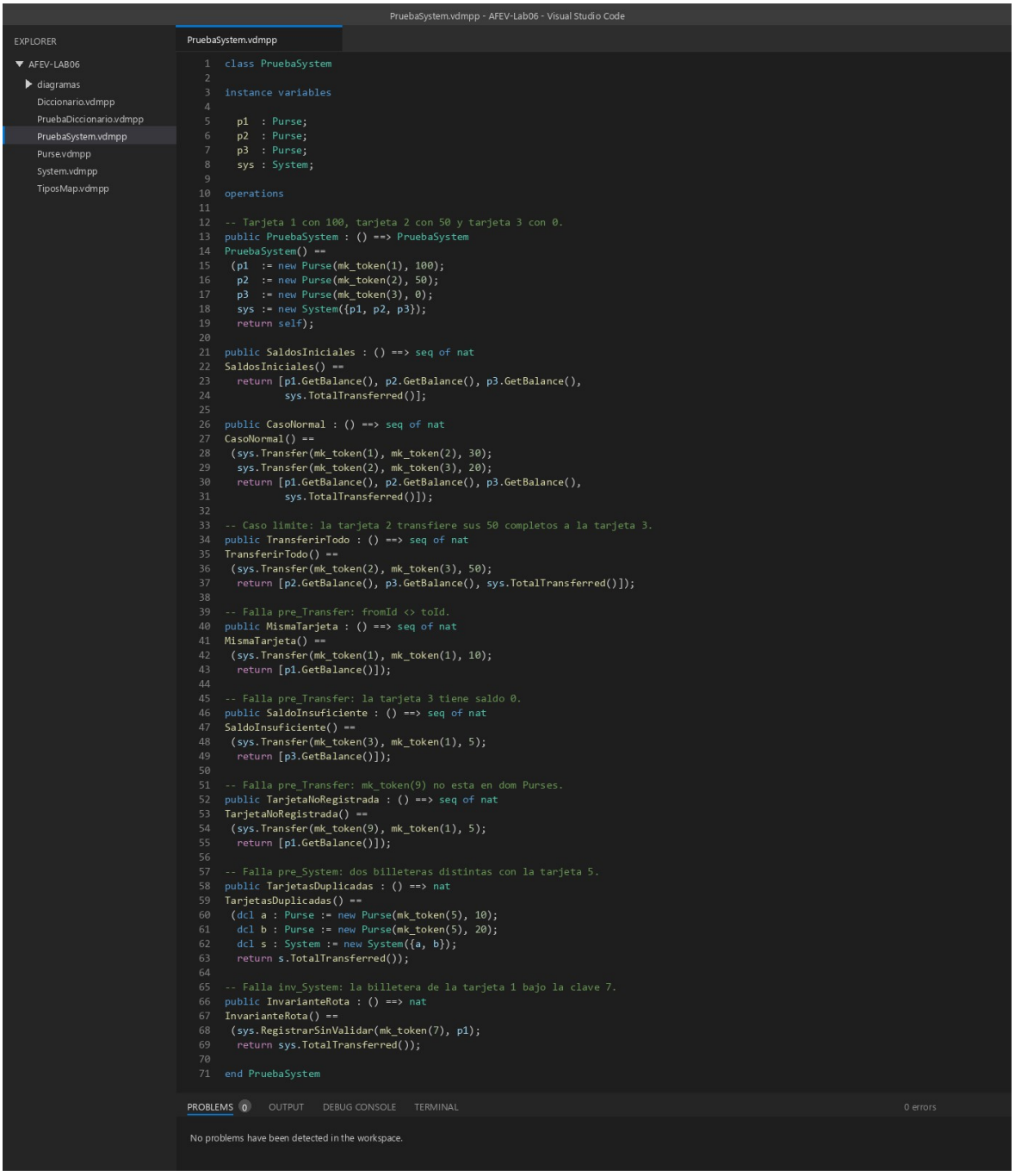


Parte 1 de System.vdmpp: el map Purses con su invariante, la bitácora Log, el registro

Transaction, el constructor con la comprensión de mappings y la operación Transfer con su precondición.



Parte 2 de System.vdmpp: TotalTransferred, la función recursiva TotalSum y la operación RegistrarSinValidar, marcada como solo para pruebas.



PruebaSystem crea en su constructor tres billeteras (tarjeta 1 con 100, tarjeta 2 con 50 y tarjeta 3 con 0) y un System. Cada operación ejecuta un caso de prueba sobre un objeto nuevo.

3. Operaciones del tipo Map utilizadas (inciso a)

Las líneas corresponden a System.vdmpp, que conserva la misma disposición del código de la guía.

Línea	Operación (Tabla 1 / Tabla 2)	Uso en el código
3	Tipo map (map D to R)	Declara Purses : map Purse`CardId to Purse.
4	dom m	forall p in set dom Purses recorre todas las tarjetas registradas.
4	Aplicación m(d)	Purses(p) obtiene la billetera guardada bajo la clave p.
15	Comprensión de mappings (Tabla 1)	{p.GetCardNo() |-> p | p in set PurseSet} construye el map en el constructor.
20, 21	Aplicación m(d)	Purses(fromId) y Purses(toId) obtienen las billeteras para DecreaseBal e IncreaseBal.
23	dom m	{fromId, toId} subset dom Purses verifica que ambas tarjetas estén registradas.
25	Aplicación m(d)	Purses(fromId).GetBalance() consulta el saldo de origen.
42	m1 ++ m2	Solo en RegistrarSinValidar, operación de prueba que no es parte del código de la guía.

No se usan rng, munion, merge, las restricciones (<:, <-:, :>, :->) ni inverse. La comprensión no figura en la Tabla 2, sino en la Tabla 1 como constructor.

4. Invariante en lenguaje natural (inciso b)

Para toda tarjeta p registrada en Purses, la billetera guardada bajo la clave p debe tener como número de tarjeta a p. Es decir, ninguna billetera puede estar guardada bajo el número de otra tarjeta. Por ejemplo, el maplet mk_token(7) |-> (billetera de la tarjeta 1) viola la invariante.

La invariante se cumple por construcción: el constructor usa p.GetCardNo() como clave de cada maplet, y Transfer modifica los saldos pero no el map.

5. Diagrama de clases (inciso c)

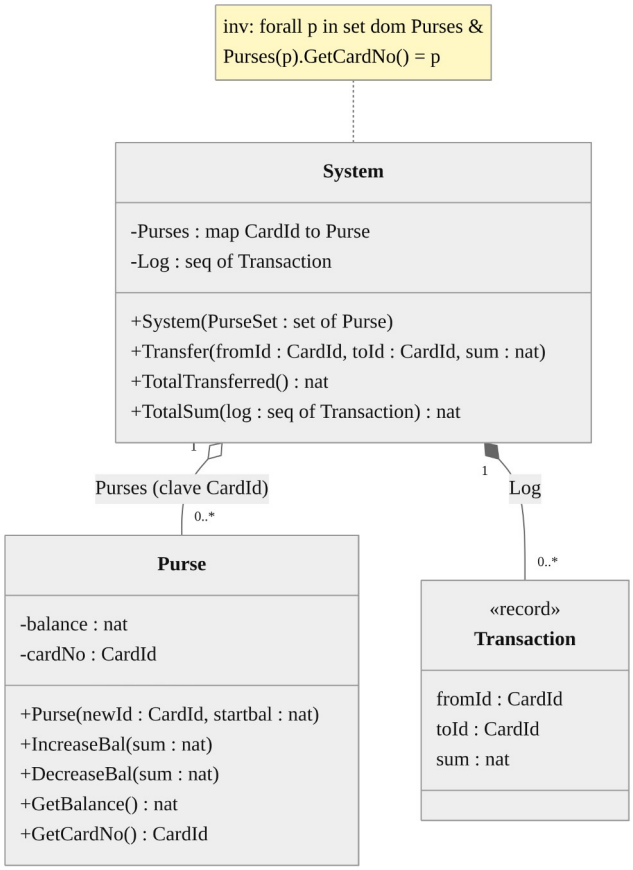


Diagrama de clases del modelo. System agrega de 0 a muchas billeteras Purse a través del map Purses, cuya clave es CardId, y compone la bitácora Log de registros Transaction. La nota muestra la invariante de System.

Se usó agregación entre System y Purse porque las billeteras se crean fuera de System y se reciben en el constructor. Transaction es un tipo registro definido dentro de System, por eso se marcó con el estereotipo «record» y como composición. La clave del map también podría dibujarse como una asociación calificada por CardId.

6. Explicación de las operaciones

Purse(newId, startbal): constructor. Asigna el número de tarjeta y el saldo inicial.

IncreaseBal(sum) y DecreaseBal(sum): suman o restan sum al saldo. DecreaseBal tiene la precondición sum <= balance, que impide un saldo negativo, ya que balance es nat.

GetBalance() y GetCardNo(): consultas que devuelven el saldo y la tarjeta. Se declararon pure porque no cambian el estado y se invocan desde la invariante y la precondición.

System(PurseSet): construye Purses por comprensión, con la tarjeta de cada billetera como clave. La precondición exige que dos billeteras distintas no compartan tarjeta; sin ella la comprensión

tendría una clave con dos valores.

Transfer(fromId, toId, sum): resta sum a la billetera de origen, lo suma a la de destino y agrega un mk_Transaction al final de Log. La precondición se evalúa de izquierda a derecha: primero confirma que las tarjetas están en dom Purses y solo después aplica Purses(fromId).

TotalTransferred() y TotalSum(log): TotalSum recorre Log de forma recursiva y suma el campo sum de cada transacción. El intérprete puede mostrar una advertencia porque la función recursiva no declara measure; no impide la ejecución.

RegistrarSinValidar(id, pu): operación solo para pruebas. Agrega el maplet id |-> pu con ++ sin comprobar la tarjeta, para provocar la violación de la invariante.

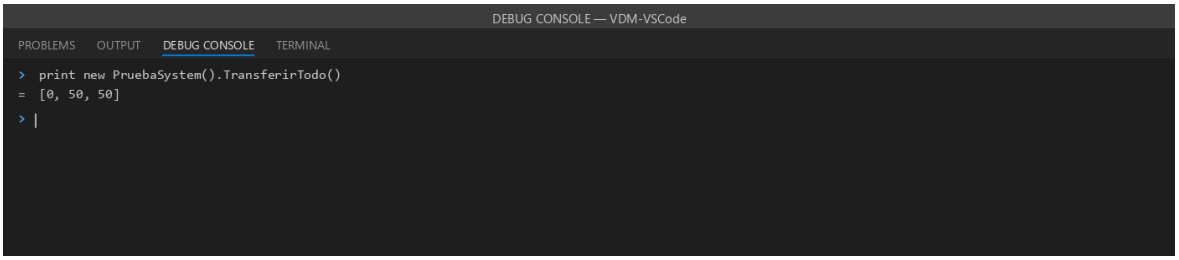
7. Comandos de validación en consola VDM

Saldos iniciales y caso normal



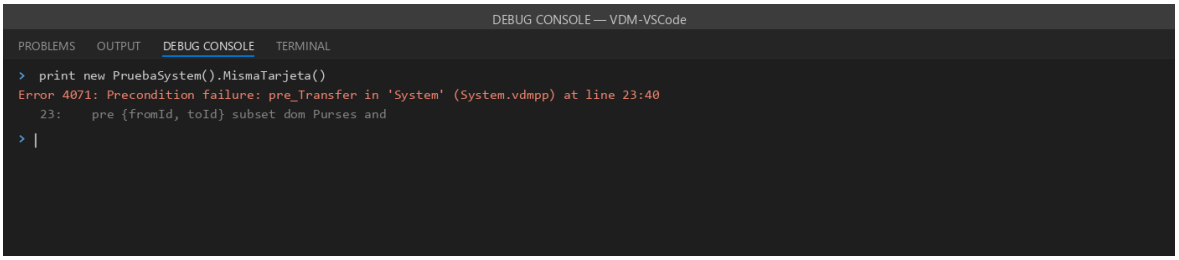
SaldosIniciales devuelve [100, 50, 0, 0]. CasoNormal transfiere 30 de la tarjeta 1 a la 2 y 20 de la 2 a la 3, y devuelve [70, 60, 20, 50]: el total de dinero se mantiene en 150 y TotalTransferred suma las dos transacciones del Log.

Caso límite: transferir todo el saldo



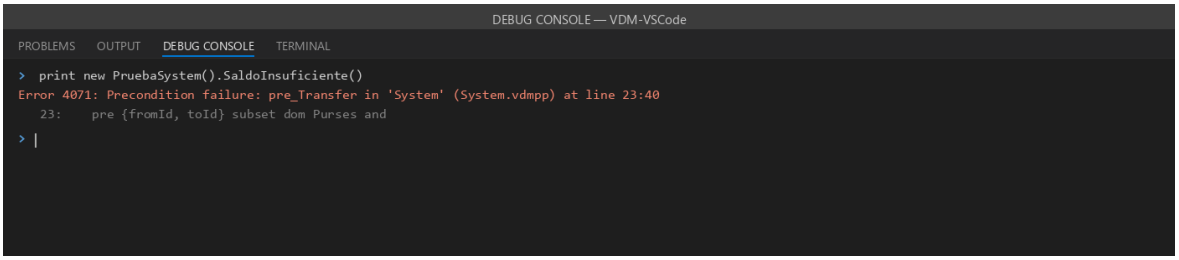
Devuelve [0, 50, 50]. La precondición usa >=, por lo que se permite transferir el saldo completo y dejar la billetera en 0.

Violación de precondición: misma tarjeta de origen y destino



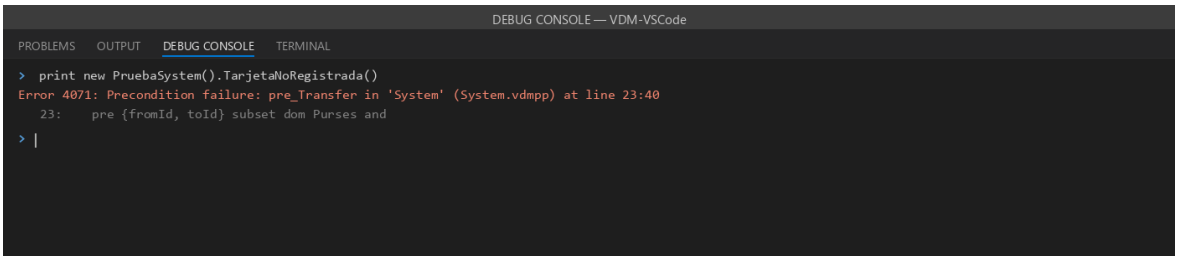
fromId <> toId no se cumple y el intérprete reporta una falla de la precondición de Transfer antes de modificar los saldos.

Violación de precondición: saldo insuficiente



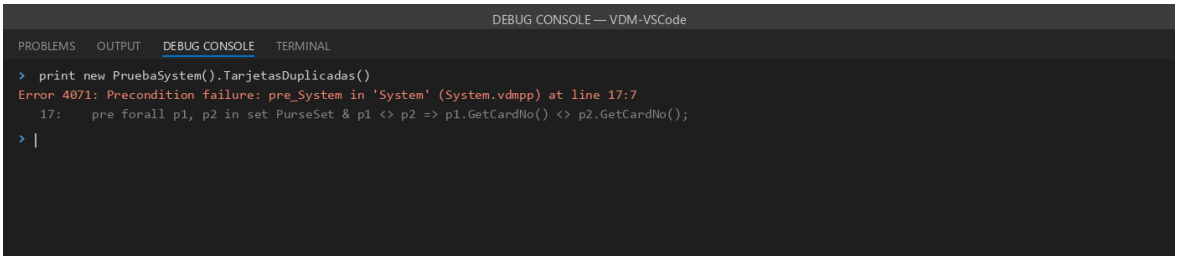
La tarjeta 3 tiene saldo 0 y se intenta transferir 5. Falla la condición Purses(fromId).GetBalance() >= sum.

Violación de precondición: tarjeta no registrada



mk_token(9) no está en dom Purses y falla {fromId, toId} subset dom Purses. Esta condición evita que Transfer aplique el map con una clave inexistente.

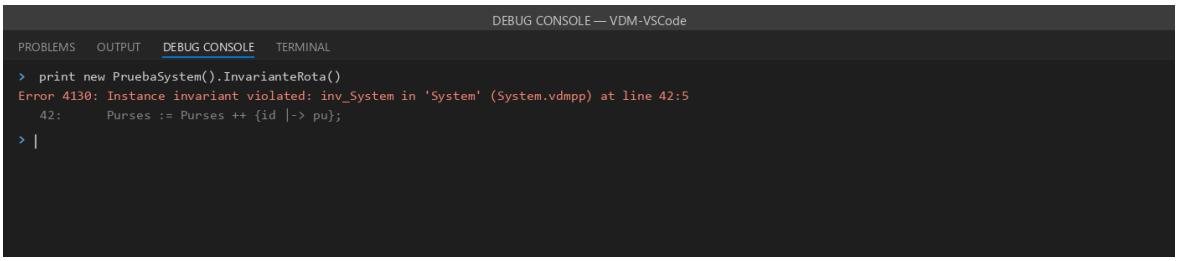
Violación de precondición del constructor: tarjetas repetidas



Se crean dos billeteras distintas con la tarjeta 5. Falla la precondición del constructor System y el

objeto no se crea.

Violación de invariante de System



RegistrarSinValidar guarda la billetera de la tarjeta 1 bajo la clave 7. Purses(7).GetCardNo() es 1, distinto de 7, y el intérprete reporta violación de la invariante de System.

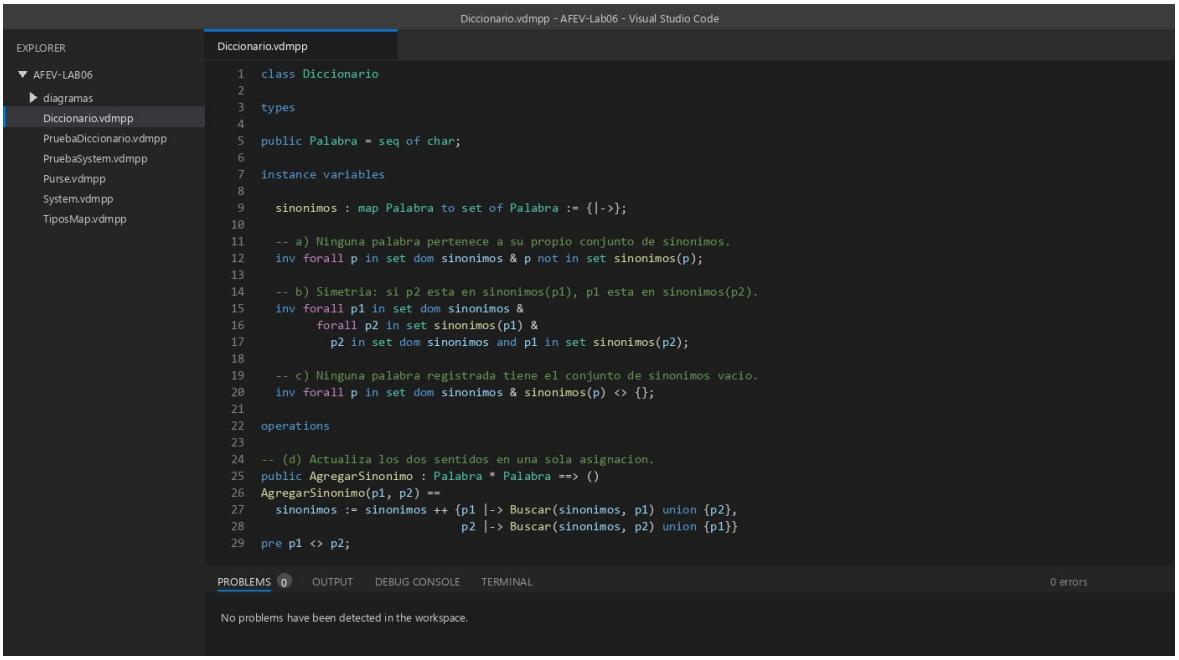
Ejercicio 2: Diccionario de Sinónimos. Caso de aplicación: clase Diccionario

1. Descripción del caso

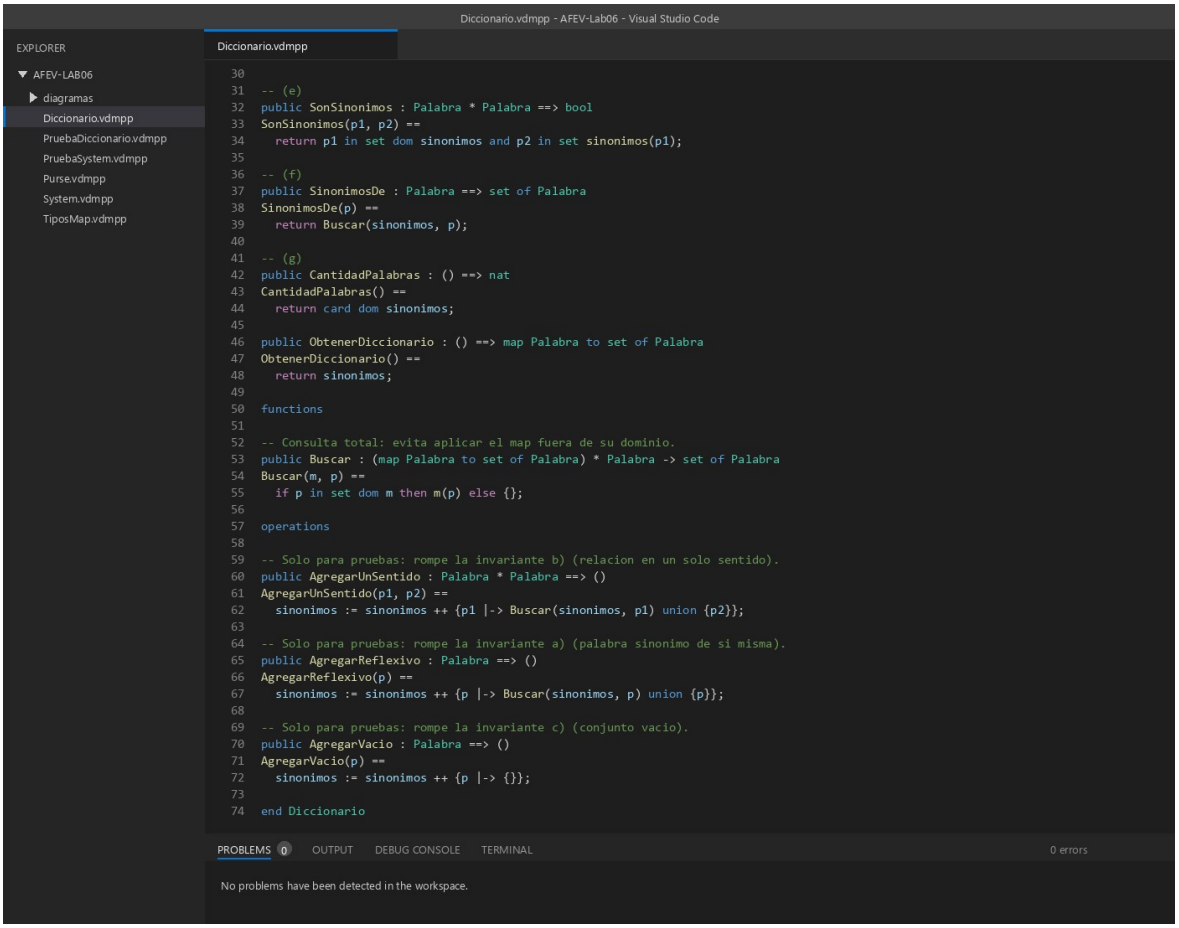
Un diccionario asocia a cada palabra el conjunto de sus sinónimos mediante sinonimos : map Palabra to set of Palabra, con Palabra = seq of char. La relación es simétrica y ninguna palabra es sinónimo de sí misma. En la guía el tipo Palabra aparece antes de class Diccionario; en VDM++ los tipos deben declararse dentro de una clase, por lo que se movió a Diccionario.

- a) Ninguna palabra pertenece a su propio conjunto de sinónimos.
- b) Simetría: si p2 está en sinonimos(p1), p1 está en sinonimos(p2).
- c) Ninguna palabra registrada tiene el conjunto de sinónimos vacío.
- Operaciones: AgregarSinonimo (d), SonSinonimos (e), SinonimosDe (f) y CantidadPalabras (g).
- Se añaden AgregarUnSentido, AgregarReflexivo y AgregarVacio, solo para pruebas, que omiten las validaciones para demostrar que cada invariante detecta su error. La clase PruebaDiccionario agrupa los casos.

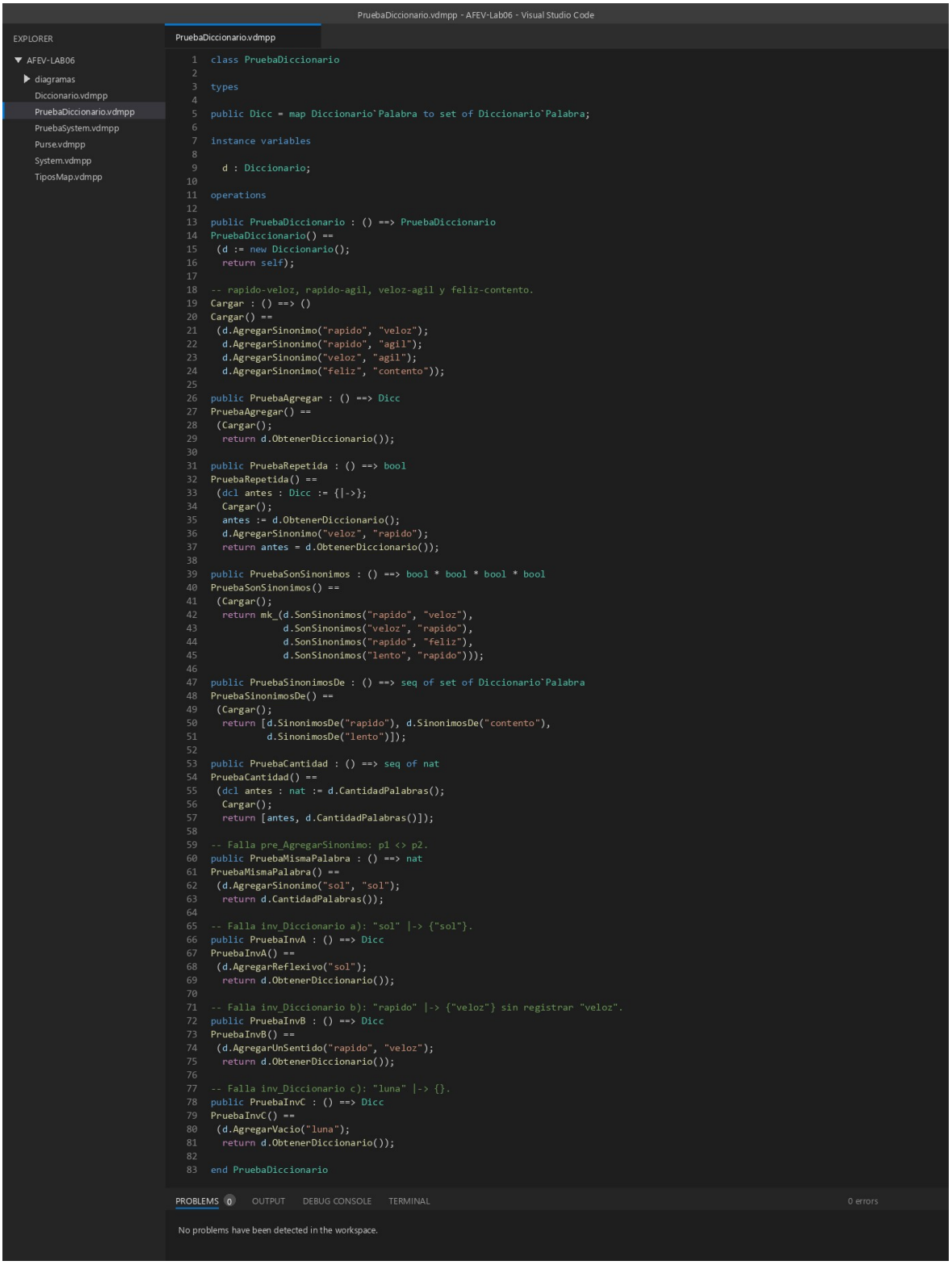
2. Código completo - Diccionario.vdmpp



Parte 1 de Diccionario.vdmpp: el tipo Palabra, el map sinonimos inicializado vacío, las invariantes (a), (b) y (c) y la operación AgregarSinonimo con su precondición.



Parte 2 de Diccionario.vdmpp: las operaciones de consulta (e, f, g), la función auxiliar Buscar y las tres operaciones solo para pruebas. La extensión no reporta errores.



PruebaDiccionario parte de un diccionario vacío. Cargar registra las relaciones rapido-veloz,

rapido-agil, veloz-agil y feliz-contento, y cada operación pública ejecuta un caso de prueba.

3. Explicación de las operaciones

Invariante (a): forall p in set dom sinonimos & p not in set sinonimos(p). Revisa cada palabra registrada y exige que no esté en su propio conjunto.

Invariante (b): dos forall anidados: para cada palabra p1 y cada sinónimo p2 de p1, exige p2 in set dom sinonimos and p1 in set sinonimos(p2). La primera parte es necesaria, porque aplicar sinonimos(p2) con p2 fuera del dominio no está definido; and se evalúa de izquierda a derecha y no llega a esa aplicación.

Invariante (c): forall p in set dom sinonimos & sinonimos(p) <> {}. Una palabra solo está en el dominio si tiene al menos un sinónimo.

Buscar(m, p): función auxiliar. Devuelve m(p) si p está en dom m y {} en caso contrario. Es una consulta total que evita aplicar el map fuera de su dominio.

AgregarSinonimo(p1, p2) (d): con ++ actualiza en una sola asignación los dos maplets: p1 recibe Buscar(sinonimos, p1) union {p2} y p2 recibe Buscar(sinonimos, p2) union {p1}. Así nunca existe un estado con la relación en un solo sentido. La precondición p1 <> p2 protege la invariante (a). Si la relación ya existía, union no duplica elementos y el estado no cambia.

SonSinonimos(p1, p2) (e): devuelve p1 in set dom sinonimos and p2 in set sinonimos(p1). Si p1 no está registrada devuelve false sin error.

SinonimosDe(p) (f): devuelve Buscar(sinonimos, p), es decir, el conjunto de sinónimos o {} si la palabra no está registrada.

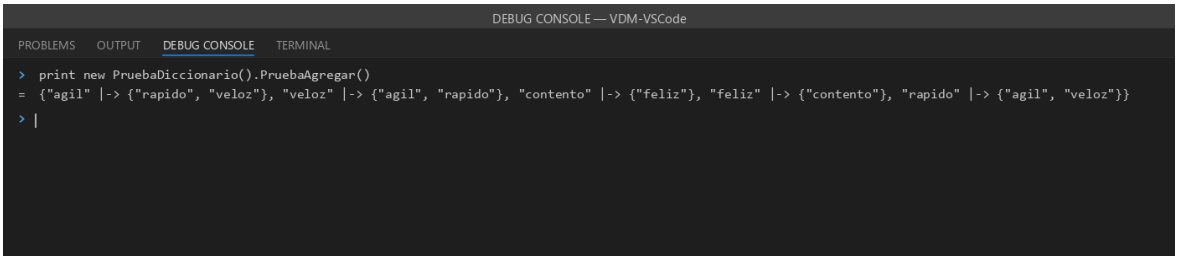
CantidadPalabras() (g): devuelve card dom sinonimos, el número de palabras registradas.

ObtenerDiccionario(): devuelve el map completo. Se usa en las pruebas para mostrar el estado.

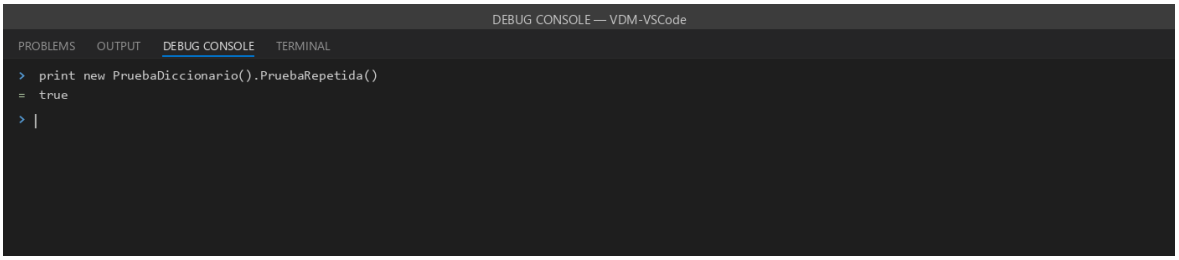
AgregarUnSentido, AgregarReflexivo y AgregarVacio: operaciones solo para pruebas. Rompen a propósito las invariantes (b), (a) y (c), respectivamente. AgregarUnSentido es la operación defectuosa de la pregunta 3 del cuestionario, con la consulta protegida por Buscar.

4. Comandos de validación en consola VDM

AgregarSinonimo (d)

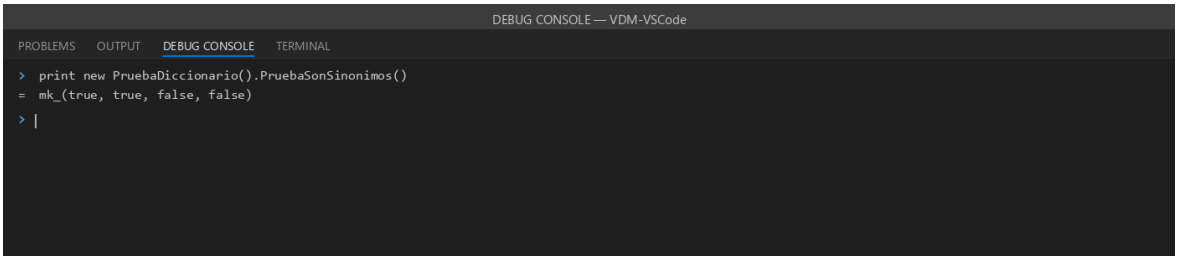


Se obtiene un map con 5 palabras: "rapido" |-> {"agil", "veloz"}, "veloz" |-> {"agil", "rapido"}, "agil" |-> {"rapido", "veloz"}, "feliz" |-> {"contento"} y "contento" |-> {"feliz"}. Cada relación aparece en ambos sentidos. El orden de los elementos puede variar.



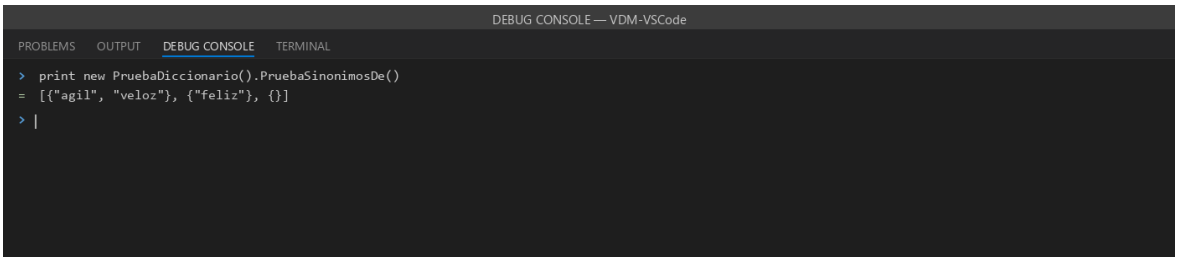
Devuelve true: volver a agregar la relación veloz-rapido, ya registrada, deja el map igual porque union no duplica elementos.

SonSinonimos (e)



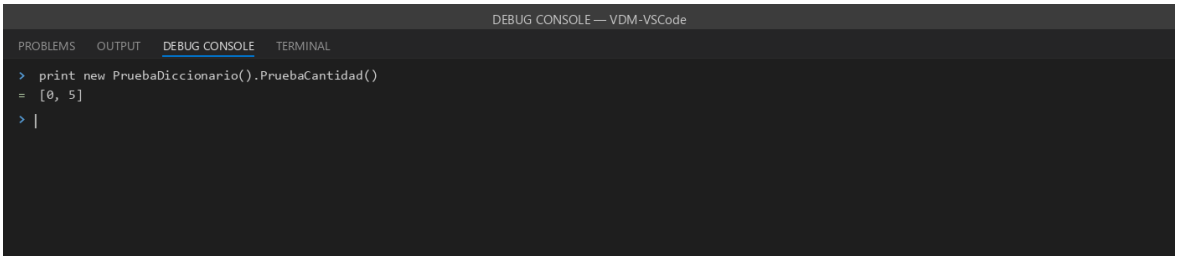
Devuelve mk_(true, true, false, false). rapido-veloz es true en ambos sentidos por la simetría, rapido-feliz es false y lento-rapido es false sin error, aunque "lento" no está registrada.

SinonimosDe (f)



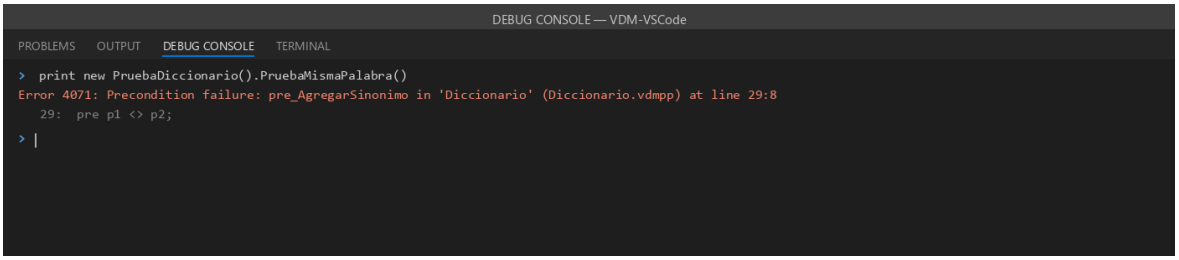
Devuelve [{"agil", "veloz"}, {"feliz"}, {}]. Para "lento", que no está registrada, se obtiene el conjunto vacío en lugar de un error.

CantidadPalabras (g)



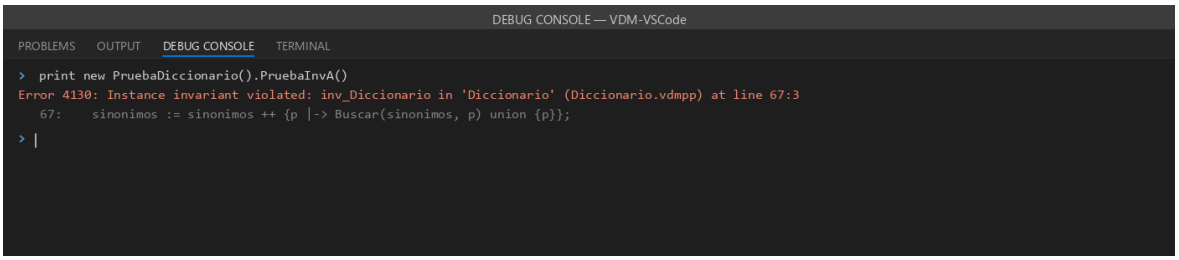
Devuelve [0, 5]: 0 palabras con el diccionario vacío y 5 después de cargar las cuatro relaciones (rapido, veloz, agil, feliz y contento).

Violación de precondición: palabra sinónimo de sí misma



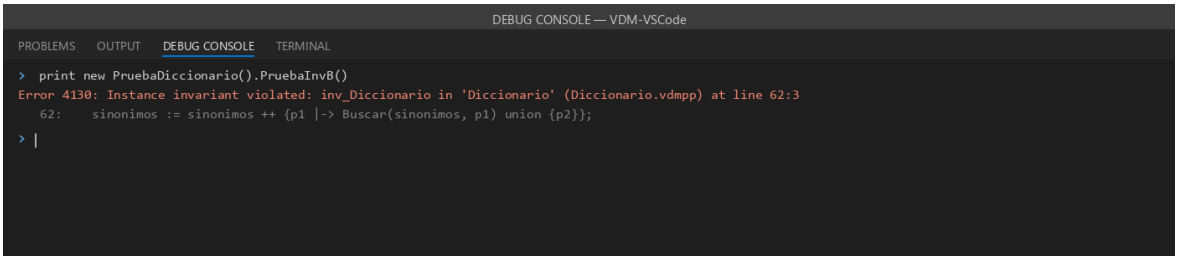
AgregarSinonimo("sol", "sol") no cumple p1 <> p2 y el intérprete reporta una falla de precondición sin modificar el diccionario.

Violación de invariante a) palabra en su propio conjunto



AgregarReflexivo deja "sol" |-> {"sol"}. La simetría y el conjunto no vacío se cumplen, pero falla la invariante (a) y se reporta violación de invariante.

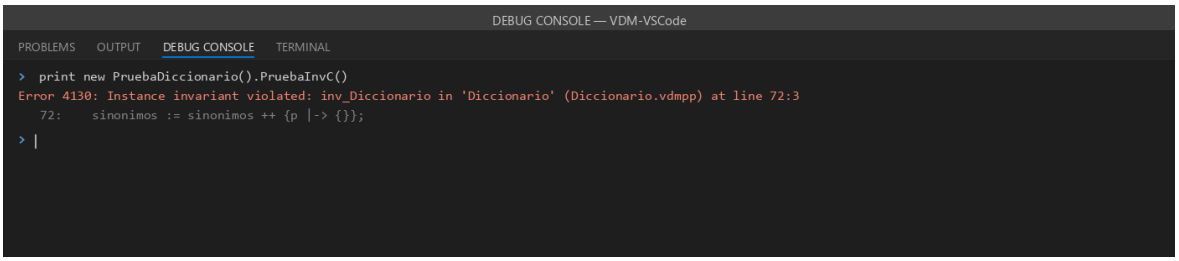
Violación de invariante b) relación en un solo sentido



AgregarUnSentido deja "rapido" |-> {"veloz"} sin registrar "veloz". Falla la invariante (b), lo que

muestra por qué AgregarSinonimo actualiza ambos sentidos en la misma asignación.

Violación de invariante c) conjunto de sinónimos vacío



AgregarVacio deja "luna" |-> {}. Las invariantes (a) y (b) se cumplen, pero falla la (c) porque la palabra está en el dominio sin sinónimos.

III. CUESTIONARIO

1. Piense en un sistema real o inventado por usted (distinto a Purse/System o Diccionario de Sinónimos) que tenga un Map relevante. Proponga UNA invariante para ese map usando 'dom', 'rng' o ambos, y explique en sus propias palabras qué problema del mundo real evita esa invariante y qué pasaría si no existiera.

Sistema: el estacionamiento de un centro comercial. Cada espacio ocupado se asocia con la placa del vehículo que lo usa.

```
instance variables
  espacios   : set of EspacioId;
  asignacion : map EspacioId to Placa;
inv dom asignacion subset espacios and
    card rng asignacion = card dom asignacion
```

La parte con dom impide asignar un vehículo a un espacio que no existe en el plano. La parte con rng exige que no haya placas repetidas en el rango, es decir, que un mismo auto no ocupe dos espacios a la vez. Sin la invariante, el sistema podría marcar ocupado el espacio 120 en un edificio de 100 espacios, o registrar la placa V2X-581 en los espacios 3 y 15. En ambos casos el conteo de espacios libres y el cobro por tiempo quedarían mal. La segunda condición equivale a declarar asignacion como inmap.

2. Proponga dos mappings propios m1 y m2 que compartan AL MENOS una clave con valores DISTINTOS asociados (por ejemplo, los precios de dos tiendas para algunos productos en común). Explique qué ocurre si evalúa 'm1 munion m2' en ese caso, por qué 'm1 ++ m2' sí produce un resultado, y justifique cuál de los dos preferiría usar en su propio ejemplo y por qué.

Uso dos listas de precios (en soles) de bodegas del mismo barrio:

```
tiendaA = {"arroz" |-> 4.5, "azucar" |-> 3.8, "aceite" |-> 9.9}
```

```
tiendaB = {"arroz" |-> 4.2, "leche" |-> 3.5}
```

Ambos mapas tienen la clave "arroz" con valores distintos (4.5 y 4.2). munion solo está definido si las claves comunes tienen el mismo valor, así que tiendaA munion tiendaB produce un error en tiempo de ejecución. ++ siempre está definido: tiendaA ++ tiendaB = {"arroz" |-> 4.2, "azucar" |-> 3.8, "aceite" |-> 9.9, "leche" |-> 3.5}, porque gana el mapa de la derecha. tiendaB ++ tiendaA daría "arroz" |-> 4.5, por lo que ++ no es conmutativo.

En mi ejemplo prefiero munion si el objetivo es unir dos catálogos que deberían coincidir, porque el error avisa que hay precios inconsistentes en lugar de ocultarlo. Usaría ++ solo si tiendaB fuera una lista actualizada que reemplaza a la anterior. Si lo que se busca es el precio más bajo, ninguno de los dos sirve directamente y se necesita una comprensión:

```
tiendaA ++ tiendaB ++
  {p |-> (if tiendaA(p) < tiendaB(p) then tiendaA(p) else tiendaB(p))
   | p in set dom tiendaA inter dom tiendaB}
```

3. Se le presenta la siguiente operación de una clase cuya invariante exige que 'sinonimos' sea SIMÉTRICO (si p2 está en sinonimos(p1), entonces p1 debe estar en sinonimos(p2)):

```
public AgregarSinonimo: Palabra * Palabra ==> ()
AgregarSinonimo(p1, p2) ==
  sinonimos := sinonimos ++ {p1 |-> sinonimos(p1) union {p2}};
```

a) Identifique DOS errores de especificación en esa operación (piense en qué pasa con la simetría, y en qué pasa si p1 todavía no está en 'dom sinonimos').

Error 1: solo actualiza p1. Agrega p2 a sinonimos(p1), pero no agrega p1 a sinonimos(p2), y se rompe la simetría. Si p2 no estaba registrada, ni siquiera queda en dom sinonimos. Error 2: si p1 no está en dom sinonimos, la expresión sinonimos(p1) aplica el map fuera de su dominio y la operación falla en tiempo de ejecución, justo la primera vez que se registra la palabra. Además, sin precondición, AgregarSinonimo(p, p) dejaría a p como sinónimo de sí misma.

b) Proponga la corrección completa y justifique, comparando con cómo el Ejercicio Resuelto evitó un problema similar.

```
public AgregarSinonimo: Palabra * Palabra ==> ()
AgregarSinonimo(p1, p2) ==
  sinonimos := sinonimos ++ {p1 |-> Buscar(sinonimos, p1) union {p2},
                             p2 |-> Buscar(sinonimos, p2) union {p1}}
pre p1 <> p2;

-- Buscar(m, p) == if p in set dom m then m(p) else {}
```

Una sola asignación con dos maplets evita que exista un estado con la relación en un solo sentido. Buscar devuelve {} si la palabra no está registrada, así no se aplica el map fuera de su dominio. pre p1 <> p2 protege la invariante (a) y evita que el map por extensión tenga la misma clave con dos valores. En el Ejercicio Resuelto no hay una operación equivalente; lo más cercano es la Parte 3, donde ++ se prefiere porque siempre está definido, mientras munion falla con claves en conflicto.

La corrección sigue esa idea: usar solo expresiones definidas para cualquier entrada. El mismo cuidado aparece en Transfer, que verifica {fromId, toId} subset dom Purses antes de aplicar Purses(fromId). La prueba PruebaInvB reproduce el error 1.

4. ¿En qué situación usted preferiría un Map (map D to R) en lugar de un conjunto de pares (set of (D * R))? ¿Y en qué situación al revés? Proponga un ejemplo propio (no usado en esta guía) para cada caso y explique qué garantía perdería si usara la estructura equivocada.

- Map: el padrón de electores, DNI → datos del elector. Cada DNI identifica a una sola persona y la consulta principal es por clave, padron(dni). Con set of (DNI * Elector) podrían coexistir mk_("40123456", ana) y mk_("40123456", luis), y se perdería la garantía de no ambigüedad, salvo que se agregue una invariante a mano.
- Conjunto de pares: la matrícula alumno–curso. Un alumno lleva varios cursos y un curso tiene varios alumnos, es una relación de muchos a muchos. Con map AlumnoId to CursoId cada alumno solo podría tener un curso, y al registrar el segundo con ++ se perdería el primero sin aviso. map AlumnoId to set of CursoId también funciona, pero el conjunto de pares permite consultar por cualquiera de los dos lados de forma directa.

5. De todo lo trabajado en esta práctica (constructores, mappings válidos / no inyectivos / inválidos, dom, rng, munion, ++, invariantes), ¿cuál idea le costó más entender al principio? Explique, con sus propias palabras y un ejemplo suyo, qué fue lo que finalmente le ayudó a entenderla o qué duda le queda todavía.

Lo que más me costó fue diferenciar un mapping no inyectivo de uno inválido. Al principio pensé que {3 |-> <Apple>, 4 |-> <Apple>} estaba mal porque <Apple> se repetía, como si todo map tuviera que ser uno a uno. Lo entendí con un ejemplo de notas: {"Ana" |-> 15, "Luis" |-> 15} es válido porque dos alumnos pueden tener la misma nota, pero {"Ana" |-> 15, "Ana" |-> 12} no, porque no se sabe cuál es la nota de Ana. La regla es que una clave no puede apuntar a dos valores; un valor sí puede repetirse. También ayudó ver en la consola que rng devuelve {<Apple>} una sola vez. Me queda la duda de cuándo conviene declarar inmap en lugar de map con una invariante como card rng m = card dom m.

IV. CONCLUSIONES

1. El tipo Map modela asociaciones en las que cada clave tiene exactamente un valor, como un índice o una tabla con clave primaria. Con dom, rng, la aplicación m(d), munion, ++ y la restricción <: se consultan y combinan estas asociaciones sin recorridos manuales, y la comprensión permite construirlas a partir de una regla.

2. En el ejercicio resuelto se comprobó que munion es parcial y falla cuando una clave común tiene valores distintos, mientras que ++ siempre está definido porque el mapa de la derecha sobrescribe. También se vio que rng es un conjunto, por lo que los valores repetidos de un mapping no inyectivo se cuentan una sola vez, y que un filtro que ningún elemento cumple produce el mapping vacío.

3. En los ejercicios propuestos, la combinación de forall ... in set dom con la aplicación m(d)

expresó reglas sobre todo el dominio: en System, que cada billetera esté guardada bajo su propia tarjeta; en Diccionario, la simetría, la no reflexividad y la ausencia de conjuntos vacíos. AgregarSinonimo mantuvo la simetría al actualizar ambos sentidos en una sola asignación con + +.

4. La aplicación m(d) solo está definida si d pertenece a dom m. Por eso la invariante de simetría verifica p2 in set dom sinonimos antes de aplicar sinonimos(p2), y se definió Buscar para devolver {} en lugar de fallar. También se encontraron partes de la guía que no son coherentes con los tipos declarados, como el tipo Palabra fuera de la clase o los literales <England> y <Spain>, que no pertenecen a PAIS.

5. En el trabajo profesional, un map con invariantes cumple el papel de las restricciones de integridad de una base de datos: unicidad de claves, consistencia entre la clave y el dato guardado y relaciones simétricas. Especificarlas antes de programar permite validar requisitos y detectar con el intérprete estados inválidos que en el código final serían errores difíciles de rastrear.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

[1] Fitzgerald, J., Larsen, P. G., Mukherjee, P., Plat, N., & Verhoef, M. (2005). Validated designs for object oriented systems. Springer Science & Business Media.

[2] Baier, C., & Katoen, J. P. (2008). Principles of model checking. MIT Press.

[3] Overture Tool. (2026). Overture Tool Documentation. Recuperado de https://www.overturetool.org/

[4] VDMTools. (2026). VDMTools User Guide. Recuperado de https://github.com/vdmtools/vdmtools

[5] VDM Portal. (2026). The Vienna Development Method. Recuperado de http://www.vdmportal.org/